ALLIANCE UNIVERSITY • SCHOOL OF COMPUTING • DEPARTMENT OF COMPUTER APPLICATIONS

Assignment 6: Implementation of Data Integrity Constraints

Relational Constraints, Business Rules & Rigorous SQL Verification Test Suite

Student Name: Sanjeev Adarsh S
Program / Semester: BCA • Semester III
Course: Database Management Systems (DBMS Lab)
Deliverable: Individual Assignment 6 (PDF)

1. Introduction to Relational Data Integrity

In database systems, **data integrity** refers to the overall accuracy, completeness, and consistency of stored data. Without declarative integrity constraints, application bugs, concurrent updates, network faults, and malicious user inputs will inevitably corrupt database state, introducing orphaned records, negative inventories, and contradictory demographics.

This assignment presents the complete implementation of integrity constraints across the **Smart Library Management System (SLMS)**. It categorizes constraints across the four classical relational integrity pillars, explains the operational rationale for each rule, and demonstrates empirical verification through both positive test cases and negative test cases that trigger explicit database rejection errors.

2. The Four Pillars of Relational Data Integrity

Integrity Category	Relational Definition & SQL Mechanism	SLMS Operational Implementation
Entity Integrity	Ensures that every tuple in a relation is uniquely identifiable and that no component of a primary key contains a `NULL` value. Implemented via `PRIMARY KEY` constraints.	Every table possesses a designated primary key: surrogate keys for master tables (e.g., `book_id`, `member_id`) and a composite key for the junction table `(book_id, author_id)`.
Referential Integrity	Guarantees that relationships between tables remain consistent. A foreign key in a child table must either match a candidate key of a parent table or be wholly `NULL`. Implemented via `FOREIGN KEY` and referential actions.	Links books to categories and publishers, loans to books, members, and staff, and fines to loans with explicit `ON DELETE RESTRICT` and `ON DELETE CASCADE` rules.
Domain Integrity	Restricts the valid range, format, and data type of values permitted within an attribute column. Implemented via SQL data types, `NOT NULL`, and `CHECK` expressions.	Enforces numerical ranges (`price ≥ 0.0`, `1500 ≤ publication_year ≤ 2026`) and validates date boundaries across all circulation records.
User-Defined Integrity	Enforces organization-specific business rules and institutional policies not covered by fundamental relational theory. Implemented via `CHECK` enums, `UNIQUE`, and `DEFAULT` values.	Restricts membership types (`'Student'`, `'Faculty'`, `'Alumni'`, `'Guest'`), operational staff roles, ensures `available_copies ≤ total_copies`, and sets automated default dates.

3. Comprehensive Constraint Architecture Matrix

The table below details all declarative constraints embedded directly into the DDL architecture of SLMS:

Table	Constraint Type	SQL Declaration	Purpose & Failure Protection
`publishers`	`UNIQUE` `DEFAULT`	`publisher_name UNIQUE` `country DEFAULT 'United States'`	Prevents duplicate publisher records; populates standard publishing hub country automatically.
`categories`	`UNIQUE`	`category_name UNIQUE`	Eliminates redundant shelf categories and taxonomy confusion in the catalog.
`books`	`UNIQUE` `CHECK (Year)` `CHECK (Copies)` `CHECK (Price)`	`isbn UNIQUE` `CHECK(year BETWEEN 1500 AND 2026)` `CHECK(avail ≥ 0 AND avail ≤ total)` `CHECK(price ≥ 0.0)`	Ensures ISBN uniqueness globally; blocks invalid historical or future publication years; prevents negative inventory and phantom checkouts; blocks negative retail pricing.
`book_authors`	`PRIMARY KEY` `FOREIGN KEY`	`PRIMARY KEY (book_id, author_id)` `ON DELETE CASCADE`	Prevents duplicate author associations on the same title; purges orphaned junction entries if a book or author is removed.
`members`	`UNIQUE` `CHECK (Type)` `CHECK (Status)` `DEFAULT`	`email UNIQUE, phone UNIQUE` `CHECK(type IN ('Student','Faculty','Alumni','Guest'))` `CHECK(status IN ('Active','Suspended','Expired'))` `DEFAULT CURRENT_DATE`	Blocks duplicate accounts; restricts patron categories to authorized tiers; limits lifecycle status; stamps registration date automatically.
`staff`	`CHECK (Role)` `UNIQUE`	`CHECK(role IN ('Head Librarian',...))` `email UNIQUE`	Restricts administrative privileges to valid library operational roles; enforces unique staff login credentials.
`loans`	`FOREIGN KEY` `CHECK (Status)` `DEFAULT`	`FK → books, members, staff (RESTRICT)` `CHECK(status IN ('Issued','Returned','Overdue','Lost'))` `issue_date DEFAULT CURRENT_DATE`	Prevents deletion of active members or books with outstanding checkouts; enforces valid circulation workflow transitions; stamps checkout date.
`fines`	`UNIQUE` `CHECK (Amount)` `CHECK (Paid)` `CHECK (Status)`	`loan_id UNIQUE` `CHECK(fine_amount ≥ 0.0)` `CHECK(paid ≥ 0.0 AND paid ≤ fine_amount)` `CHECK(status IN ('Unpaid','Partially Paid','Paid'))`	Enforces strict 1:1 penalty mapping per delinquent loan; blocks negative fine amounts; prevents overpayment beyond assessable debt; restricts payment state transitions.

4. Empirical Constraint Verification Test Suite

To demonstrate that the database engine actively enforces these integrity rules, a comprehensive test suite was executed against the database. The suite evaluates both valid positive transactions and illegal negative mutations.

4.1 Positive Test Cases (Normal Valid Operations)

Valid lifecycle data inserts conforming strictly to data types, keys, and check expressions execute successfully with zero errors:

```
-- Positive Test 1: Insert valid publisher, category, and book
INSERT INTO publishers (publisher_id, publisher_name, country, contact_email)
VALUES (1, 'MIT Press', 'United States', 'info@mitpress.mit.edu');

INSERT INTO categories (category_id, category_name, description)
VALUES (1, 'Computer Science & AI', 'Foundational algorithms and computing theory');

INSERT INTO books (book_id, isbn, title, category_id, publisher_id, publication_year, total_copies,
available_copies, price)
VALUES (1, '978-0262033848', 'Introduction to Algorithms (4th Ed)', 1, 1, 2022, 8, 8, 89.99);

-- Positive Test 2: Insert valid member and loan
INSERT INTO members (member_id, full_name, email, phone, membership_type)
VALUES (1, 'Sanjeev Adarsh', 'sanjeev.a@alliance.edu.in', '+91-9876543210', 'Student');

INSERT INTO staff (staff_id, staff_name, role, email)
VALUES (1, 'David Miller', 'Head Librarian', 'dmiller@smartlibrary.org');

INSERT INTO loans (loan_id, book_id, member_id, staff_id, due_date)
VALUES (1, 1, 1, 1, '2026-11-01');

[STATUS]: All positive statements executed and committed successfully. Entity and referential
integrity intact.
```

4.2 Negative Test Case 1: UNIQUE ISBN Constraint Violation

Objective: Verify that the database rejects the insertion of a book possessing an ISBN already registered to an existing title.

```
-- Negative Test 1 SQL: Attempt to insert duplicate ISBN '978-0131103627'
INSERT INTO books (isbn, title, category_id, publisher_id, publication_year, total_copies,
available_copies, price)
VALUES ('978-0131103627', 'Duplicate ISBN Book', 1, 1, 2021, 3, 3, 40.00);
```

```
sqlite3.IntegrityError: UNIQUE constraint failed: books.isbn
[TEST RESULT: PASSED • Database successfully blocked duplicate ISBN registration]
```

4.3 Negative Test Case 2: CHECK Constraint Violation on Inventory Copies

Objective: Verify that the business rule `available_copies ≤ total_copies` is strictly defended by the database kernel.

```
-- Negative Test 2 SQL: Attempt to insert available_copies=5 when total_copies=3
INSERT INTO books (isbn, title, category_id, publisher_id, publication_year, total_copies,
available_copies, price)
VALUES ('978-0131109999', 'Invalid Copies Book', 1, 1, 2020, 3, 5, 50.00);
```

```
sqlite3.IntegrityError: CHECK constraint failed: available_copies >= 0 AND available_copies <= total_copies
[TEST RESULT: PASSED • Database blocked physically impossible inventory state]
```

4.4 Negative Test Case 3: Foreign Key Constraint Violation (Orphan Category Reference)

Objective: Verify that books referencing a non-existent category identifier are rejected, preserving referential consistency.

```
-- Negative Test 3 SQL: Reference non-existent category_id=999
INSERT INTO books (isbn, title, category_id, publisher_id, publication_year, total_copies,
available_copies, price)
VALUES ('978-0131108888', 'Invalid FK Book', 999, 1, 2020, 4, 4, 30.00);
```

```
sqlite3.IntegrityError: FOREIGN KEY constraint failed
[TEST RESULT: PASSED • Database prevented orphaned title record]
```

4.5 Negative Test Case 4: Domain Enumeration CHECK Constraint Violation

Objective: Verify that patron membership types are restricted to authorized institutional tiers.

```
-- Negative Test 4 SQL: Attempt to insert unauthorized membership_type 'SuperAdmin'
INSERT INTO members (full_name, email, phone, membership_type)
VALUES ('Unauthorized User', 'bad@test.com', '1234567890', 'SuperAdmin');
```

```
sqlite3.IntegrityError: CHECK constraint failed: membership_type IN ('Student', 'Faculty', 'Alumni',
'Guest')
[TEST RESULT: PASSED • Database rejected invalid privilege escalation and illegal domain value]
```

5. Summary of Constraint Verification Results

Test Identifier	Target Constraint	Induced Failure Mechanism	Verification Status
Integrity Test 1	`UNIQUE (isbn)`	Duplicate catalog edition registration	**PASSED (Rejected)**
Integrity Test 2	`CHECK (copies)`	Available copies exceeding total physical inventory	**PASSED (Rejected)**
Integrity Test 3	`FOREIGN KEY (category_id)`	Referencing non-existent category key	**PASSED (Rejected)**
Integrity Test 4	`CHECK (membership_type)`	Unauthorized membership enumeration string	**PASSED (Rejected)**

> ***Conclusion & Impact:*** *By implementing primary keys, foreign keys, unique constraints, check conditions, and default values directly in the SQL DDL layer, the Smart Library Management System guarantees that data integrity is enforced natively by the relational engine. Even in the event of client-side application bugs, concurrent race conditions, or direct API manipulation, the database kernel strictly defends entity, referential, domain, and user-defined integrity.*